

Hoja de Ruta Técnica

PAI-1 · IntegriDos — Security Team (Grupo 20)

Plan por fases y pasos —no por fechas— para cada rol y para el equipo en conjunto: cada rol avanza a su ritmo, y el orden de los pasos (y sus dependencias) marca cuándo tiene sentido empezar cada uno. Cada paso indica qué construir, de qué depende y cómo saber que está terminado.

-  **Rol A** — Credenciales y acceso (RS1 + RF1)
-  **Rol B** — Integridad y anti-replay (RS2 + RS3 + RF2)
-  **Rol C** — Canal lateral y evidencias (RS4 + docs)
-  **Conjunto** — puntos de sincronización de los 3 roles

Vista general — fases del equipo

	Fase 1 Diseño y contrato	Fase 2 Implementación base	Fase 3 Endurecimiento y protecciones	Fase 4 Integración y pruebas de ataque	Fase 5 Documentación	Fase 6 Revisión final y entrega
ROL A	Modelo usuario + contrato de mensaje	Argon2id + /register + /login	Rate limiting + /logout	Tests + soporte a integración	Memoria: RS1	Revisión y entrega
ROL B	Contrato de mensaje + esqueleto HMAC	sign/verify HMAC + middleware	Nonces + /transfer	Tests + soporte a integración	Memoria: RS2/RS3	Revisión y entrega
ROL C	Entorno Wireshark + captura baseline	Captura tráfico legítimo	Auditoría timing + experimento	Ataques MitM/Replay	Diagramas + memoria RS4	Revisión y entrega
CONJUNTO	Kickoff: contrato del mensaje			Integración end-to-end	Matriz de trazabilidad	Checklist + empaquetado

Rol A y Rol B pueden arrancar la Fase 2 en paralelo en cuanto cierran juntos la Fase 1: el **contrato del mensaje** (qué campos lleva el JSON, qué cabeceras HTTP se usan y cómo se deriva la clave de sesión). Si eso no se fija antes, cada rol construye piezas que luego no encajan. Rol C puede avanzar su Fase 1 y parte de la Fase 2 (entorno de captura, tráfico legítimo) en paralelo con A y B, pero su auditoría de tiempo constante (Fase 3) no tiene sentido empezarla hasta que Rol B tenga el HMAC funcionando.

1**Modelo de datos de usuario y contrato del mensaje**

Fase 1

- Definir la tabla/estructura de usuario: username (clave), hash de contraseña, IBAN asociado.
- Reunión conjunta con Rol B: acordar el formato exacto del JSON de transacción y las cabeceras (X-Signature, X-Nonce, X-Timestamp) para que ambos roles construyan sobre el mismo contrato.
- Decidir cómo se deriva la clave de sesión (256 bits) que Rol B usará para firmar: se genera en el login y se entrega al cliente junto al token de sesión.

✓ *Criterio de aceptación: El equipo tiene un documento (aunque sea una página compartida) con la estructura exacta del JSON y las cabeceras, firmado por A y B antes de escribir código.*

2**Derivación segura de contraseñas (Argon2id)**

Fase 2

- Implementar una función que derive un hash Argon2id con salt aleatorio único por usuario (la librería lo genera y lo embebe automáticamente en el hash).
- Implementar la función de verificación correspondiente.
- Elegir y documentar los parámetros de coste (iteraciones, memoria, paralelismo): un equilibrio entre seguridad y tiempo de respuesta razonable en un portátil.

✓ *Criterio de aceptación: Registrar dos veces la misma contraseña produce dos hashes distintos (por el salt), y ambos verifican correctamente contra la contraseña original.*

3**Endpoint de registro (RF1.a, RF1.c)**

Fase 2

- Endpoint que recibe username + password (+ IBAN) y crea el usuario.
- Rechazar explícitamente el registro si el username ya existe.
- Nunca devolver ni loguear la contraseña en texto plano en ningún punto del flujo.

✓ *Criterio de aceptación: Un segundo intento de registrar el mismo username devuelve un error claro (409), sin sobrescribir al usuario existente.*

4**Endpoint de login y apertura de sesión (RF1.d)**

Fase 2

- Verificar credenciales con la función del paso 2.
- Generar un token de sesión y la clave de sesión de 256 bits (PRNG criptográfico, no random.random()).
- Cuidado con timing: verificar la contraseña también cuando el usuario no existe (contra un hash 'dummy'), para no filtrar por tiempo de respuesta si un username existe o no.

✓ *Criterio de aceptación: Login correcto devuelve token + clave de sesión; login con usuario inexistente y login con contraseña incorrecta tardan aproximadamente lo mismo en responder.*

5

Rate limiting / bloqueo temporal (RS1.b)

Fase 3

- Contador de intentos fallidos consecutivos por username.
- Bloqueo temporal (p. ej. 60 segundos) al superar el umbral (p. ej. 5 intentos).
- El contador se resetea tras un login correcto.

✓ *Criterio de aceptación: Tras N intentos fallidos seguidos, un login con la contraseña CORRECTA también es rechazado hasta que pase el tiempo de bloqueo.*

6

Logout y datos de prueba (RF1.b, RF1.d)

Fase 3

- Endpoint de logout que invalida el token de sesión.
- Script de arranque que precarga 2-3 usuarios de prueba (para que Rol B y Rol C puedan probar sin tener que registrarse cada vez).

✓ *Criterio de aceptación: Tras logout, ese token de sesión ya no sirve para operaciones autenticadas.*

7

Tests unitarios de la parte de credenciales

Fase 4

Depende de: Pasos 1-6 completados

- Test: registrar dos veces el mismo usuario falla.
- Test: login con credenciales correctas funciona; con incorrectas, falla.
- Test: N fallos consecutivos bloquean la cuenta; tras el tiempo de bloqueo, vuelve a aceptar.
- Test: dos hashes de la misma contraseña son distintos entre sí.

✓ *Criterio de aceptación: La suite de tests de Rol A pasa en verde de forma reproducible (pytest).*

1**Contrato del mensaje firmado (con Rol A)**

Fase 1

- Misma reunión del paso 1 de Rol A: fijar juntos el JSON de la transacción, las cabeceras X-Signature/X-Nonce/X-Timestamp, y qué se firma exactamente (payload + nonce + timestamp, no solo el payload).
- Decidir la serialización canónica del JSON antes de firmar (mismo orden de claves y mismo formato en cliente y servidor — si no, la firma nunca coincidirá).

✓ *Criterio de aceptación: Documento compartido con la estructura exacta, aceptado por A y B.*

2**Módulo de firma HMAC-SHA256**

Fase 2

- Función que genera la clave secreta con un PRNG criptográfico (mínimo 256 bits).
- Función que firma un mensaje: serializa de forma canónica, concatena con nonce y timestamp, y calcula el HMAC-SHA256.
- Función de verificación equivalente, que recalcula la firma esperada y la compara con la recibida usando comparación en tiempo constante (esto es RS4, pero conviene dejarlo bien desde el principio en vez de arreglarlo después).

✓ *Criterio de aceptación: Firmar el mismo mensaje dos veces con la misma clave da la misma firma; cambiar un solo carácter del payload cambia la firma completamente.*

3**Middleware de verificación en el servidor**

Fase 2

Depende de: Paso 2 (módulo HMAC)

- Punto único (dependencia/middleware) que se ejecuta antes de cualquier lógica de negocio: extrae las cabeceras, valida timestamp, valida firma.
- Diseñarlo para ser reutilizable en cualquier endpoint que reciba un mensaje firmado, no solo en /transfer.

✓ *Criterio de aceptación: Un mensaje sin cabeceras de seguridad, o con una firma inválida, es rechazado antes de tocar la base de datos de transacciones.*

4**Nonces y ventana de timestamp (RS3)**

Fase 3

Depende de: Paso 3 (middleware)

- Generación de un nonce aleatorio e impredecible por cada mensaje, en el cliente.
- Tabla/registro en el servidor de nonces ya procesados, con purga de los que ya salieron de la ventana temporal válida.
- Definir y documentar la ventana de validez del timestamp (p. ej. 30 segundos).

✓ *Criterio de aceptación: Reenviar exactamente el mismo mensaje firmado una segunda vez es rechazado por nonce repetido, aunque la firma sea perfectamente válida.*

5

Endpoint de transferencia (RF2)

Fase 3

Depende de: Pasos 2-4

- El servidor solo registra la transacción tras pasar el middleware de verificación completo (firma + nonce + timestamp).
- Idempotencia: si por lo que sea llega dos veces un tx_id ya procesado (más allá del control de nonce), no debe duplicar el efecto.

✓ *Criterio de aceptación: Una transacción válida y nueva se registra correctamente; la misma transacción reenviada no se procesa dos veces.*

6

Tests de integridad y anti-replay

Fase 4

Depende de: Pasos 1-5 completados

- Test: firma válida es aceptada.
- Test: payload alterado tras firmar es rechazado (simula MitM).
- Test: clave incorrecta es rechazada.
- Test: nonce repetido es rechazado.
- Test: timestamp fuera de ventana es rechazado.

✓ *Criterio de aceptación: La suite de tests de Rol B pasa en verde de forma reproducible.*

7

Checkpoint de integración con Rol C

Fase 4

- Exponer a Rol C un cliente/script de ejemplo que firma y envía mensajes correctamente, para que pueda construir sus escenarios de ataque sobre una base que sí funciona.
- Confirmar con Rol C qué endpoints y formato exacto va a auditar en el paso de tiempo constante.

✓ *Criterio de aceptación: Rol C puede generar tráfico legítimo y capturarlo sin ayuda de Rol B.*

1**Entorno de captura de tráfico**

Fase 1

- Instalar y probar Wireshark (o tcpdump) en la máquina donde correrá el servidor.
- Preparar el filtro de captura (por puerto, p. ej. `tcp.port == 8080`) y comprobar que se ve tráfico de prueba antes de que exista ninguna protección.
- Acordar con Rol A/B el nombre y ubicación de los ficheros .pcap dentro del repositorio.

✓ *Criterio de aceptación: Una captura de prueba (aunque sea de un 'hola mundo' HTTP) se abre correctamente en Wireshark y se filtra bien por puerto.*

2**Captura de tráfico legítimo (baseline)**

Fase 2

- En cuanto Rol A/B tengan login y transfer mínimamente funcionando (aunque sea sin toda la seguridad), capturar un flujo completo: login + transferencia aceptada.
- Esta captura sirve como referencia visual de 'cómo se ve el tráfico normal' antes de mostrar los ataques.

✓ *Criterio de aceptación: Captura .pcap guardada en el repositorio con un login y una transferencia legítima completos.*

3**Auditoría de comparaciones en tiempo no constante**

Fase 3

Depende de: Rol B - paso 2 (módulo HMAC)

- Revisar TODO el código (servidor y cliente) buscando comparaciones de HMACs, tokens o firmas con el operador de igualdad normal.
- Confirmar con Rol B que la función de verificación de firmas usa una comparación en tiempo constante, y no una comparación ingenua.

✓ *Criterio de aceptación: Ninguna comparación de HMAC/token en el proyecto usa el operador de igualdad normal; todas pasan por la función seleccionada de comparación en tiempo constante.*

4**Experimento de análisis de tiempo constante**

Fase 3

- Diseñar un script que mida el tiempo de verificación de una firma correcta frente a firmas incorrectas que difieren en distintas posiciones de byte.
- Repetir cada medición muchas veces (varios cientos/miles) y quedarse con la mediana, para filtrar ruido del sistema operativo.
- Comparar dos implementaciones: una comparación ingenua (vulnerable) y la función en tiempo constante usada realmente en el proyecto.

✓ *Criterio de aceptación: Se obtiene una tabla o gráfica que muestra que el tiempo de la comparación ingenua varía según dónde está el primer byte incorrecto, y el de la función en tiempo constante no.*

5

Escenario de ataque Man-in-the-Middle

Fase 4

Depende de: Rol B - paso 5 (endpoint transfer)

- Construir un escenario donde un mensaje legítimo se intercepta y se modifica en tránsito (por ejemplo, cambiar el importe) antes de llegar al servidor.
- Capturar el intercambio completo con Wireshark: el mensaje original, la modificación, y el rechazo del servidor (código de error).

✓ *Criterio de aceptación: Captura .pcap donde se ve un mensaje alterado y la respuesta de rechazo del servidor, lista para incluir en la memoria.*

6

Escenario de ataque Replay

Fase 4

Depende de: Rol B - paso 4 (nonces)

- Capturar un mensaje legítimo ya procesado y reenviarlo tal cual una segunda vez.
- Capturar el intercambio con Wireshark: el envío original aceptado y el reenvío rechazado por nonce repetido.

✓ *Criterio de aceptación: Captura .pcap con el mensaje duplicado y la respuesta de rechazo del servidor.*

7

Diagramas de secuencia y matriz de trazabilidad

Fase 5

- Diagramas de secuencia cliente-servidor: login, transferencia legítima, ataque MitM rechazado, ataque Replay rechazado.
- Coordinar con A y B la matriz de trazabilidad: qué fichero/línea implementa cada requisito (RS1-RS4, RF1-RF2).

✓ *Criterio de aceptación: Los 4 diagramas de secuencia y la matriz de trazabilidad están listos para pegar en la memoria.*

Hoja de ruta conjunta

Puntos donde los tres roles se sincronizan obligatoriamente. Si alguno de estos hitos se atasca mucho más que el resto, es la señal para reorganizar tareas o pedir ayuda antes de que bloquee al resto del equipo.

1

Kickoff + contrato del mensaje

Fase 1

- Los tres roles fijan juntos: formato del JSON de transacción, cabeceras HTTP, cómo se deriva la clave de sesión, y estructura de carpetas del repositorio.
- Es también el seguimiento con el profesor: llevar al menos la estructura y el contrato de mensaje decidido, aunque el código esté vacío.

✓ *Criterio de aceptación: Documento de contrato de mensaje compartido y accesible para los tres roles.*

2

Primer bloque en paralelo

Fase 2

- Rol A y Rol B implementan su parte de forma independiente sobre el contrato ya fijado, sin bloquearse mutuamente.
- Rol C prepara su entorno de captura y toma la primera evidencia de tráfico en cuanto haya algo mínimamente funcional.

✓ *Criterio de aceptación: Al final del bloque: login + registro funcionando (Rol A) y firma/verificación HMAC funcionando de forma aislada (Rol B), aunque no estén aún conectados.*

3

Segundo bloque: RS3 y RS4 arrancan

Fase 3

Depende de: Sprint 1

- Rol B conecta el middleware de verificación al endpoint de transferencia y añade nonces.
- Rol C empieza su auditoría de tiempo constante en cuanto el HMAC de Rol B esté operativo.
- Rol A cierra su parte con tests y rate limiting.

✓ *Criterio de aceptación: Un flujo completo login → transferencia firmada → aceptada funciona de punta a punta al menos una vez.*

4

Integración end-to-end y revisión cruzada

Fase 4

Depende de: Pasos 1-3

- Prueba conjunta del flujo completo: registro, login, transferencia legítima, ataque MitM rechazado, ataque Replay rechazado.
- Revisión cruzada de código: cada rol revisa el de otro buscando TLS colado, comparaciones inseguras o hashes obsoletos.
- Rol C captura los escenarios de ataque ya con el sistema completo integrado.

✓ *Criterio de aceptación: Las capturas .pcap de MitM y Replay están hechas sobre el sistema final, no sobre una versión provisional.*

5

Redacción de la memoria conjunta

Fase 5

- Resumen ejecutivo, diseño del protocolo, manual de despliegue (entre todos).
- Cada rol redacta el apartado de análisis de seguridad de su propio requisito, con sus propias evidencias ya generadas.
- Apartado de uso de IA: qué tareas se apoyaron en asistentes de IA y cómo se validó el resultado.

✓ *Criterio de aceptación: Borrador completo de la memoria, dentro del límite de 10 páginas.*

6

Checklist de cumplimiento y revisión final

Fase 6

- Repasar en grupo: cero TLS/HTTPS/SSL, cero MD5/SHA1, cero comparaciones inseguras, claves de 256 bits, servidor rechaza estrictamente antes de procesar.
- Revisar que la memoria no supera 10 páginas y que el .zip incluye código, logs y evidencias.
- Dejar margen antes del plazo de entrega de la asignatura: no dejar el empaquetado y la subida para el último momento.

✓ *Criterio de aceptación: Checklist completo marcado en verde por los tres roles.*

7

Empaquetado y entrega

Fase 6

- Generar PAI1-ST20.zip con código fuente, logs/evidencias y documentación.
- Subir exclusivamente por la Plataforma de Enseñanza Virtual, en la actividad correspondiente (nunca por email ni mensajería interna: penalización del 10%).

✓ *Criterio de aceptación: Confirmación de entrega recibida, con margen respecto al plazo de la asignatura.*